

Formation Claude

Ali Koudri - ali.koudri@janus-academy.fr



Ali Koudri

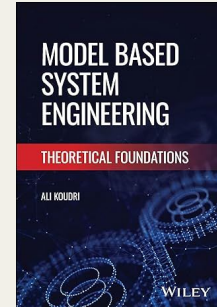
- PhD en informatique
- 25+ ans en ingénierie système et logicielle, modélisation, performance, sécurité
- IRT SystemX - Thales - CEA
- Janus Academy : formation & conseil indépendant
- Approche cross-stack : front, back, ci/cd, observabilité

Publications

Model Based Systems Engineering: Theoretical Foundations

Posture pédagogique

Démarche systémique, mesure avant optimisation, capitalisation transposable



Agenda

- Prise en main
- Configuration et personnalisation
- Sub-agents et équipes d'agents
- Sécurité
- Environnements reproductibles
- Aller plus loin

Support et labs : <https://github.com/akoudri/claude-training.git>

Repo git à cloner



MODULE M0

Claude Code en perspective

Claude & Claude Code

Motivation · histoire · philosophie · positionnement

Ce que couvre M0

1

Motivation

Ce que l'autocomplétion ne résout pas

2

Histoire

De projet interne à outil largement adopté

3

Philosophie

Les principes de conception qui expliquent ses choix

4

Positionnement

Face à Cursor, Copilot, Windsurf, Codex CLI

Pas d'atelier ici : l'objectif est de comprendre le pourquoi avant d'attaquer le comment en M1.



MOTIVATION

Ce que l'autocomplétion ne résout pas

Trois limites d'un assistant classique

Portée

Une suggestion à la fois, ligne par ligne — pas une tâche qui touche plusieurs fichiers

Exécution

Aucun accès aux commandes, aux tests, au terminal : c'est à vous de vérifier

Initiative

Chaque suggestion attend une validation — pas de boucle autonome jusqu'au résultat

Pourquoi un agent plutôt qu'un assistant

Un agent explore, planifie, exécute et vérifie jusqu'au résultat — la boucle vue en M1. Ce n'est pas une meilleure autocomplétion, c'est un mode de collaboration différent.

À garder en tête

Le reste de cette formation part de ce constat : la valeur n'est pas dans la vitesse de frappe, mais dans la capacité à déléguer une tâche entière.



HISTOIRE

De projet interne à outil largement adopté

HISTOIRE

Une trajectoire rapide

Sept. 2024

Démarre comme projet interne chez Anthropic, porté par un ingénieur

Fév. 2025

Research preview, aux côtés de Claude 3.7 Sonnet

Mai 2025

Disponibilité générale, aux côtés de Claude Opus 4 et Sonnet 4

Depuis

Adoption d'abord interne chez Anthropic, puis large — extensions IDE, web, équipes d'agents



PHILOSOPHIE

Les principes derrière les choix de conception

Composable, comme un outil Unix

Claude Code fait une chose et la fait bien : lire, exécuter, modifier. Il communique en texte, s'enchaîne avec `grep`, `jq`, `sed` — plutôt que d'imposer une interface fermée.

C'est ce choix qui rend possible ce qui a été montré en M1 et M5 : terminal, CI, scripts — le même outil partout.

« Faire simple d'abord »

Mémoire

Un fichier markdown auto-chargé (CLAUDE.md) plutôt qu'une base de connaissances complexe

Recherche dans le code

grep et glob — des outils Unix vieux de décennies — plutôt qu'une recherche vectorielle sophistiquée

Le pari : les briques les plus simples, compréhensibles et extensibles, l'emportent sur l'échafaudage sophistiqué.

Conçu pour le modèle de demain

Plutôt que de compenser les limites du modèle actuel par des règles rigides, l'outil est pensé pour un modèle plus capable, quelques mois plus tard.

Conséquence pratique

Moins de scripts rigides autour du modèle, plus de confiance dans sa capacité à utiliser des outils simples correctement.



POSITIONNEMENT

Face au reste du paysage

Des choix d'architecture différents

Cursor

IDE complet avec complétion et chat intégrés à l'éditeur

GitHub Copilot

Complétion native, forte intégration GitHub et Actions

Windsurf

IDE agentique, flux de travail intégré à l'éditeur

OpenAI Codex CLI

CLI concurrente, écrite en Rust pour la performance

Quand l'un plutôt que l'autre

Un IDE agentique intégré

Pertinent quand l'équipe veut tout au même endroit : édition, complétion, chat, dans un seul outil.

Un agent CLI composable

Pertinent quand l'agent doit s'intégrer partout — terminal, CI, scripts — sans dépendre d'un éditeur particulier.

Sur des cas d'usage réels, pas sur du dogme : les deux familles d'outils continuent d'évoluer et de se rapprocher.



MODULE M1

Prise en main de l'agent

Claude & Claude Code

Fil rouge : Argos

Ce que couvre M1

- | | | |
|---|-------------------------------|---|
| 1 | Positionnement | Boucle agentique, ce qui distingue Claude Code d'un assistant classique |
| 2 | Installation | Prérequis, méthodes, authentification |
| 3 | Anatomie d'une session | Permissions, checkpoints, .claude/, CLAUDE.md |
| 4 | Interfaces | Panorama rapide des points d'entrée disponibles |

+ Lab : installation complète et première session guidée sur Argos

Argos

Un petit gestionnaire de tickets (API + interface + tests), volontairement imparfait. Chaque module de cette formation le fait progresser : un bug à corriger, des conventions à poser, une fonctionnalité à coordonner à plusieurs agents, une faille à corriger, un environnement à reproduire, une dernière fonctionnalité construite en TDD.

Corriger un bug



Poser des
conventions



Coordonner des
agents



Corriger une faille



Reproduire
l'environnement



Construire en TDD

À la fin de ce module

- Situer Claude Code dans la boucle agentique et par rapport à un assistant IA classique
- Installer et authentifier Claude Code sur son poste
- Lire l'anatomie d'une session : permissions, checkpoints, .claude/
- Choisir l'interface adaptée à son contexte de travail
- Avoir corrigé le premier bug d'Argos avec Claude Code



BLOC 1.1

Positionnement

La boucle agentique



Cette boucle tourne jusqu'à ce que le résultat soit vérifié, pas jusqu'à ce que le code soit écrit.

Ce qui change par rapport à Copilot / Cursor

Assistant classique

Portée

Complétion locale, ligne par ligne

Initiative

Suggestion à valider en continu

Accès

Contexte de l'éditeur

Contrôle

Accepter/rejeter une suggestion

Claude Code

Autonomie sur plusieurs fichiers, jusqu'à la vérification

Explore, planifie, exécute, corrige de façon déléguée

Terminal, web, MCP, sources externes

Permissions, checkpoints, modes de fonctionnement

Ce que Claude Code peut faire

Comprendre le code

Lire, expliquer,
cartographier une base de
code existante

Exécuter

Lancer des commandes,
des tests, des scripts, dans
un environnement
contrôlé

Corriger

Diagnostiquer un échec de
test et itérer jusqu'à la
résolution

Se connecter

Accéder au web et à des
sources externes via MCP

Agent ou autocomplétion : selon le cas

Autocomplétion (Copilot/Cursor)

Modification locale, rapide, sous contrôle visuel constant ligne par ligne.

Agent (Claude Code)

Tâche à décomposer sur plusieurs fichiers, nécessitant exécution, vérification, ou recherche de contexte.

Sur des mesures et des cas réels, pas sur du dogme : les deux se combinent au quotidien.



BLOC 1.2

Installation

Prérequis et méthodes

Trois méthodes d'installation courantes ; seule la méthode npm demande Node.js :

```
# script natif (macOS / Linux / WSL) — recommandé  
curl -fsSL https://claude.ai/install.sh | bash
```

```
# gestionnaire de paquets  
brew install --cask claude-code  
winget install Anthropic.ClaudeCode
```

```
# npm (multiplateforme, Node.js 18 ou plus)  
npm install -g @anthropic-ai/claude-code
```

Vérifier l'installation : claude doctor

Authentication

- 1 Lancer claude dans un terminal
- 2 Choisir la méthode de connexion (compte Claude, Console, ou fournisseur cloud de l'entreprise)
- 3 Valider dans le navigateur ouvert automatiquement
- 4 Retour au terminal : la session est prête



BLOC 1.3

Anatomie d'une session

Permissions et checkpoints

Permissions

Chaque action sensible (édition, commande, accès réseau) est soumise à un mode de permission. Rien n'est exécuté sans que ce mode l'autorise.

Checkpoints

Chaque édition faite par les outils de Claude est suivie :
/rewind (ou Échap deux fois) revient en arrière sans passer par Git. Les effets des commandes shell ne sont pas suivis.

Le répertoire `.claude/` et `CLAUDE.md`

```
argos/  
├── CLAUDE.md  
│   # instructions et conventions du projet  
├── .claude/  
│   ├── settings.json    # permissions, préférences  
│   └── commands/        # commandes personnalisées  
└── src/ ...
```

CLAUDE.md sera repris en détail au module M2 — pour l'instant, savoir qu'il existe suffit.

Projet ou utilisateur : deux portées

.claude/ à la racine du projet

Versionné, partagé avec l'équipe, spécifique à Argos. C'est ce qu'on a montré jusqu'ici.

~/.claude/ à la racine du compte

Personnel, s'applique à tous les projets : CLAUDE.md, commandes, sub-agents et préférences valables partout.

Précédence

Les réglages projet complètent ou priment sur les réglages utilisateur, sauf les règles de refus (deny), qui l'emportent toujours, à n'importe quel niveau.

Ce qu'on configure soi-même

CLAUDE.md

Instructions personnelles, chargées à chaque session, aux côtés du CLAUDE.md du projet

settings.json

Réglages par défaut pour tous les projets : permissions, modèle, hooks

commands/

Commandes personnelles, disponibles partout (vs .claude/commands/ du projet)

agents/

Sub-agents personnels, disponibles dans tous les projets

skills/

Skills personnels, invoqués par leur nom dans n'importe quel projet

plugins/

Marketplaces ajoutées et plugins installés (dont Superpowers, vu en M6)

Ce que Claude Code écrit lui-même

projects/

Mémoire automatique et transcripts de session, par projet

file-history/

Cliché de chaque fichier avant édition — le mécanisme derrière les checkpoints (M1)

teams/, tasks/

État des équipes d'agents actives et liste de tâches partagée — le mécanisme derrière M3

sessions/

Suivi des sessions en cours (avec session-env/), pour détecter les sessions concurrentes

history.jsonl

Historique des invites tapées, pour le rappel et la recherche dans le terminal

Ces fichiers ne se modifient pas à la main : ils reflètent ce que l'agent fait, pas l'inverse.

Le reste : caches et journaux

Une partie de `~/claude/` ne sert qu'au fonctionnement interne de l'outil : `backups/`, `cache/`, `debug/`, `paste-cache/`, `shell-snapshots/`, `stats-cache.json`.

À retenir

Sûrs à supprimer pour libérer de l'espace, jamais à éditer à la main.

Par honnêteté

Quelques dossiers du compte (`ide/`, `security/`, `config.json...`) ne sont pas couverts par la documentation officielle : ne pas s'y fier pour un usage réel, seulement pour comprendre qu'ils existent.

Le contexte se remplit

Chaque message envoyé inclut tout l'historique de la session. Plus une session dure, plus le contexte se charge — jusqu'à dégrader la qualité des réponses avant même d'atteindre la limite.

Repère pratique

Viser environ 60 % d'utilisation du contexte, bien avant le seuil d'environ 80 % où le compactage automatique se déclenche.

Pourquoi agir avant la limite

Au compactage automatique, le modèle résume déjà en état dégradé. Compacter tôt, avec du contexte encore clair, produit un meilleur résumé.

/compact ou /clear

/compact

Résume la conversation et repart sur ce résumé. Le fil du travail en cours est conservé.

À utiliser : en continuité, sur la même tâche.

/clear

Efface tout l'historique, sans récupération possible. CLAUDE.md se recharge, rien d'autre.

À utiliser : en changeant de tâche, sans lien avec la précédente.

Erreur fréquente : faire /clear par réflexe alors qu'on continue la même tâche — l'agent reprend alors à zéro et peut répéter un travail déjà fait.

Suivre la consommation

/cost

Dépense de la session en cours (comptes API)

/stats

Consommation cumulée (abonnements Pro/Max)

/model

Basculer de modèle en session — un modèle plus léger pour explorer, un plus capable pour le point dur

Pour un public professionnel

Vérifier régulièrement construit l'intuition de ce qui coûte cher — une session qui tourne en rond ou un contexte trop chargé, typiquement.



BLOC 1.4

Interfaces

Où faire tourner Claude Code

Terminal (CLI)

L'interface de référence,
disponible partout

VS Code / JetBrains

Extension intégrée à
l'éditeur, diff intégré

Desktop

Application autonome,
sessions en parallèle

Web

Depuis le navigateur,
sessions dans le cloud

Panorama rapide ici — approfondi au fil de la formation (agent view, Codespaces, web en M5).

Bonnes pratiques : explorer, planifier, coder

Explorer

Faire lire et résumer le code avant toute modification

Planifier

Demander un plan, le relire, l'ajuster avant exécution

Coder

Laisser l'agent exécuter le plan validé, puis vérifier



LAB M1

Premiers pas sur Argos

Brief

SOCLE — pour tous

Installer et authentifier Claude Code. Ouvrir Argos, faire expliquer le bug par l'agent, le corriger et vérifier les tests.

EXTENSION — pour aller plus loin

Reproduire la même session dans un second point d'entrée (VS Code ou JetBrains) et comparer l'expérience.

Pièges et repères

- Un plan proposé qui semble trop large : demander de le découper avant d'accepter
- Une commande bloquée par les permissions : c'est le comportement attendu, pas une panne
- Ne pas confondre correction du symptôme et correction de la cause du bug

État attendu en fin de module

- Claude Code installé et authentifié sur chaque poste
- Le bug initial d'Argos est corrigé et les tests passent
- Chacun sait nommer permissions, checkpoints, et `.claude/`

Resynchronisation avant de passer à M2 — Configuration et personnalisation.

Ce qu'il faut retenir de M1

Claude Code exécute une boucle complète — explorer, planifier, exécuter, vérifier — avec une autonomie multi-fichiers que l'autocomplétion n'a pas.

Prochaine étape — M2

Le bug d'Argos est corrigé, mais rien n'est encore consigné pour que l'agent respecte les conventions du projet. M2 pose CLAUDE.md, une commande et un hook.



MODULE M2

Configuration et personnalisation

Claude & Claude Code

Fil rouge : Argos

Ce que couvre M2

- | | | |
|---|-----------------------------------|--|
| 1 | CLAUDE.md | Rédiger un fichier de contexte projet efficace |
| 2 | Commandes personnalisées | Encapsuler une tâche répétée en une commande |
| 3 | Hooks | Automatiser des garde-fous sur le cycle de vie d'une session |
| 4 | Plugins & marketplaces | Panorama — approfondi en extension du lab |

+ Lab : consigner les conventions d'Argos — CLAUDE.md, une commande, un hook

Depuis M1

Acquis (M1)

Le bug de filtrage des statuts est corrigé, les tests passent. Mais rien n'a été consigné : un nouvel agent qui ouvrirait Argos demain devrait redécouvrir seul les conventions du projet.

Objectif de M2

Consigner une bonne fois ce que l'agent doit savoir, plutôt que le répéter à chaque session.

À la fin de ce module

- Rédiger un CLAUDE.md court, factuel, réellement utile
- Créer une commande personnalisée pour une tâche répétée
- Poser un hook simple comme garde-fou automatique
- Situer les plugins et marketplaces sans les avoir encore pratiqués en profondeur



BLOC 2.1

CLAUDE.md

Un fichier de contexte, pas une documentation

Chargé automatiquement

Lu en début de session, sans avoir à le rappeler

Partagé par l'équipe

Versionné avec le code, identique pour tous

Vivant

Mis à jour au fil des conventions qui se stabilisent

Court, factuel, sans prose

Argos

Gestionnaire de tickets. API Node/Express + front React.

Commandes

npm test # suite complète

npm run lint # avant tout commit

Conventions

Les statuts de ticket sont toujours en minuscules.

Une section par catégorie, des faits vérifiables plutôt que des intentions.

Quand CLAUDE.md grossit : rules/

Passé environ 200 lignes, CLAUDE.md perd en clarté. `.claude/rules/` permet de scinder les conventions en fichiers thématiques, chargés au démarrage ou seulement quand un fichier concerné entre en contexte.

```
# .claude/rules/tests.md
---
paths:
  - "**/*.test.js"
---
Les tests suivent le nommage devrait<comportement>.
```

Sur Argos : une règle scindée pour les tests suffirait déjà à illustrer le mécanisme, sans attendre que CLAUDE.md déborde.



BLOC 2.2

Commandes personnalisées

Anatomie d'une commande

```
# .claude/commands/audit-statuts.md
---
description: Cherche les comparaisons de statut
             sensibles à la casse
---
Parcourir le code pour repérer toute comparaison
de statut qui ne normalise pas la casse au préalable.
```

Invocation : /audit-statuts. Format désormais recommandé : une skill, .claude/skills/audit-statuts/SKILL.md.

Quand créer une commande

Bon candidat

Une tâche précise, répétée telle quelle par plusieurs personnes de l'équipe (ex. /audit-statuts sur Argos).

Mauvais candidat

Une demande ponctuelle, formulée une seule fois : autant la poser directement en prompt.



BLOC 2.3

Hooks

Automatiser un garde-fou

Un hook est un script déclenché à un moment précis du cycle de vie d'une session : avant ou après un outil, au démarrage, à l'arrêt.

PostToolUse

Après une modification : formater le code automatiquement

PreToolUse

Avant une action : bloquer l'édition d'un fichier protégé

SessionStart

À l'ouverture : rappeler une convention critique

Protéger les migrations existantes

```
// .claude/settings.json
"hooks": {
  "PreToolUse": [{
    "matcher": "Edit|Write|MultiEdit",
    "hooks": [{
      "type": "command",
      "command": "node .claude/hooks/protect-migrations.mjs"
    }]
  }]
}
```

Le script lit l'appel d'outil en JSON sur stdin et le bloque (code de sortie 2) si la migration existe déjà ; en créer une nouvelle reste possible.



BLOC 2.4

Plugins & marketplaces

Mutualiser au niveau de l'équipe

Un plugin regroupe des commandes, hooks et sub-agents partageables. Une marketplace est un dépôt de plugins installables.

```
/plugin marketplace add <dépôt>  
/plugin install <plugin>@<marketplace>
```

Approfondi en extension du lab de ce module — installation guidée d'un plugin de démonstration.

Consigner plutôt que répéter

Répéter en prompt

À refaire à chaque session, oublié dès qu'on est pressé, jamais partagé avec le reste de l'équipe.

Consigner dans CLAUDE.md

Chargé automatiquement, versionné, identique pour toute l'équipe, corrigé une fois pour toutes.



LAB M2

Consigner Argos

Brief

SOCLE — pour tous

Rédiger le CLAUDE.md d'Argos, créer la commande /audit-statuts, poser un hook qui protège les migrations.

EXTENSION — pour aller plus loin

Installer un plugin depuis une marketplace de démonstration et vérifier ce qu'il ajoute à la session.

Pièges et repères

- Un CLAUDE.md trop long finit par ne plus être lu efficacement : rester factuel et court
- Un hook trop restrictif bloque aussi les corrections légitimes : tester avant de généraliser
- Une commande qui contient une logique métier changeante vieillit mal : la réserver aux tâches stables

État attendu en fin de module

- Argos a un CLAUDE.md versionné et factuel
- La commande /audit-statuts fonctionne
- Le hook de protection des migrations est actif et testé

Resynchronisation avant de passer à M3 — Sub-agents et équipes d'agents.

Ce qu'il faut retenir de M2

Ce qui se répète mérite d'être consigné : CLAUDE.md pour le contexte, une commande pour une tâche stable, un hook pour un garde-fou automatique.

Prochaine étape — M3

Argos est bien configuré, mais une seule session ne suffira pas pour la prochaine fonctionnalité : elle touche le front et l'API en parallèle. M3 introduit les sub-agents et les équipes d'agents.



MODULE M3

Sub-agents et équipes d'agents

Claude & Claude Code

Fil rouge : Argos

Ce que couvre M3

- | | | |
|---|-----------------------------|---|
| 1 | Sub-agents | Délégation automatique vs explicite, isolation de contexte |
| 2 | Agent teams | Patterns de coordination : superviseur/workers, revue parallèle |
| 3 | Worktrees | Isoler des sessions parallèles |
| 4 | Pour aller plus loin | Messagerie inter-sessions (encart) |

+ Lab : orchestrer une équipe d'agents sur une fonctionnalité réelle d'Argos

La prochaine fonctionnalité

Acquis (M1-M2)

Argos est corrigé et configuré : CLAUDE.md, une commande, un hook de protection sont en place.

Objectif de M3

Ajouter les commentaires sur ticket : une migration, un endpoint API, une interface. Trois fronts à la fois, une session seule y suffit mal.

À la fin de ce module

- Distinguer délégation automatique et délégation explicite à un sub-agent
- Créer un sub-agent personnalisé et savoir quand l'utiliser
- Orchestrer une équipe d'agents sur une tâche réelle
- Isoler des sessions parallèles avec des worktrees
- Avoir livré la fonctionnalité de commentaires sur Argos, sans conflit de fichiers



BLOC 3.1

Sub-agents

Deux façons de déléguer

Automatique

L'agent principal choisit lui-même de déléguer une sous-tâche à un sub-agent, sans que ce soit demandé explicitement.

Explicite

On invoque directement un sub-agent nommé pour une tâche précise, à un moment choisi.

Pourquoi isoler le contexte

Un sub-agent travaille dans son propre contexte, séparé de la conversation principale. Il revient avec un résultat condensé, pas avec tout ce qu'il a exploré.

Contexte principal préservé

L'exploration volumineuse ne pollue pas le fil de la tâche en cours

Spécialisation

Un sub-agent dédié à la revue, un autre au débogage, chacun avec ses outils propres

Créer un sub-agent

```
# .claude/agents/reviewer.md
---
name: reviewer
description: Relit un diff avant de le proposer
tools: Read, Grep, Glob
---
Relire le diff fourni. Signaler toute régression,
incohérence de convention, ou test manquant.
```

Un fichier, un rôle, un jeu d'outils restreint : plus le périmètre est net, plus le sub-agent est fiable.

Quand créer un sub-agent

Bon candidat

Une tâche récurrente et isolable, à fort volume d'exploration (revue de code, recherche dans une grande base).

Mauvais candidat

Une tâche courte et ponctuelle : le coût de délégation dépasse le bénéfice d'isolation.



BLOC 3.2

Agent teams

Coordonner plusieurs agents actifs

Superviseur / workers

Un agent répartit les sous-tâches, les autres exécutent en parallèle

Revue parallèle

Plusieurs agents examinent le même travail sous des angles différents

Sur Argos : un agent migration+API, un agent interface, un agent tests — coordonnés en superviseur/workers.

Sub-agent ou équipe d'agents ?

Sub-agent

Délégation ponctuelle. Le contrôle revient à l'agent principal une fois la sous-tâche terminée.

Agent team

Plusieurs agents actifs en parallèle sur la durée de la tâche, avec coordination continue entre eux.

Démarrer une équipe d'agents

```
Décomposer la fonctionnalité "commentaires sur  
ticket" en trois agents : migration+API, interface,  
tests. Coordonner en superviseur/workers et me  
signaler tout conflit de fichier avant de fusionner.
```

Fonction expérimentale : activer CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1, puis demander explicitement une équipe — Claude Code la compose selon la demande.

Dimensionner et éviter les conflits

- Une équipe surdimensionnée pour une tâche simple coûte plus qu'elle ne rapporte
- Allouer à chaque agent un périmètre de fichiers disjoint dès le départ
- Attendre que les coéquipiers terminent avant de fusionner leurs résultats



BLOC 3.3

Worktrees

Isoler des sessions parallèles

Un worktree Git donne à chaque session sa propre copie de travail, sur sa propre branche, sans quitter le même dépôt.

Sans worktree

Plusieurs sessions sur le même répertoire se marchent dessus

Avec worktree

Chaque agent travaille dans son propre répertoire, sur sa propre branche

En pratique

```
git worktree add -b feature/comments ../argos-comments  
cd ../argos-comments && claude
```

```
# une session par worktree, aucune collision  
# de fichiers avec la session principale
```

Ou directement : `claude --worktree comments` (worktree créé dans `.claude/worktrees/`).



POUR ALLER PLUS LOIN

Messagerie inter-sessions

Faire communiquer deux sessions

Avec la messagerie inter-sessions (ListAgents, SendMessage, /list-agents), deux sessions Claude Code s'échangent des messages ; entre machines, via Remote Control.

Cas d'usage typique

Une session de fond exécute les tests pendant qu'on continue à travailler ailleurs, et signale la fin d'exécution à la session active.

Non couvert dans le lab de ce module — à explorer en autonomie si le besoin se présente.



LAB M3

Livrer les commentaires

Brief

SOCLE — pour tous

Créer un sub-agent de revue, puis orchestrer une équipe d'agents (migration+API, interface, tests) pour livrer les commentaires sur ticket.

EXTENSION — pour aller plus loin

Isoler chaque agent dans son propre worktree, et tester une notification de fin de tâche par messagerie inter-sessions.

Pièges et repères

- Deux agents qui éditent le même fichier en parallèle créent un conflit évitable : vérifier le découpage avant de lancer
- Un agent de revue qui ne reçoit pas le diff complet ne peut pas juger correctement : lui fournir le contexte nécessaire
- Ne pas fusionner avant que tous les agents aient terminé, même si l'un d'eux semble en avance

État attendu en fin de module

- Un sub-agent de revue est créé et fonctionnel sur Argos
- Les commentaires sur ticket sont livrés : migration, API, interface, tests
- Aucun conflit de fichiers entre les agents impliqués

Resynchronisation avant de passer à M4 — Sécurité.

Ce qu'il faut retenir de M3

Le sub-agent isole une sous-tâche ponctuelle ; l'équipe d'agents coordonne plusieurs fronts en parallèle ; le worktree évite que ces fronts se marchent dessus.

Prochaine étape — M4

Argos a une nouvelle fonctionnalité, construite par plusieurs agents avec des accès larges. M4 pose les garde-fous : permissions, sandboxing, réseau, credentials.



MODULE M4

Sécurité

Claude & Claude Code

Fil rouge : Argos

Ce que couvre M4

1

Permissions

Modes de fonctionnement et quand les utiliser

2

Sandboxing

Isolation du système de fichiers, chemins protégés

3

Sécurité réseau

Domaines autorisés/bloqués, proxy

4

Credentials & MCP

Protection des secrets, sécurité des serveurs MCP

+ Lab : sandbox restrictif et allowlist réseau sur Argos, correction d'une faille

Une fonctionnalité livrée, une exposition non vue

Acquis (M1-M3)

Les commentaires sur ticket sont livrés par trois agents coordonnés, relus, testés.

Ce que personne n'a encore regardé

La clé d'API du service de notification est écrite en clair dans le code, et les agents ont accédé librement à n'importe quel domaine du réseau.

À la fin de ce module

- Choisir le mode de permission adapté à la situation
- Comprendre l'isolation filesystem et protéger des chemins sensibles
- Restreindre l'accès réseau à une liste de domaines autorisés
- Retirer une clé API du code source et la masquer correctement
- Situer les risques spécifiques aux serveurs MCP



BLOC 4.1

Permissions

Six modes de fonctionnement

plan

Analyse et propose, mais ne modifie rien sans validation explicite

par défaut

Demande confirmation pour chaque action sensible

acceptEdits

Accepte automatiquement les éditions, demande pour le reste

auto

Un classifieur relit chaque action et ne bloque que les actions risquées

dontAsk

N'exécute que les outils pré-autorisés, refuse le reste sans demander

bypassPermissions

N'importe quelle action est exécutée sans confirmation

Quand utiliser quel mode

`plan / acceptEdits`

Exploration d'un code sensible, ou itération rapide sur du code non critique déjà couvert par des tests.

`bypassPermissions`

Jamais sur du code de production. Réservé à un environnement jetable, entièrement isolé.

Au-dessus du projet : les politiques managées

managed-settings.json est déployé par l'IT, au niveau du système, hors de portée d'un projet ou d'un utilisateur. Il l'emporte sur tout le reste — le complément M1 l'avait déjà placé au sommet de la précedence.

macOS

/Library/Application Support/ClaudeCode/

Linux

/etc/claude-code/

Windows

C:\Program Files\ClaudeCode\

Ce niveau ne se configure pas depuis Argos : il vient de l'IT. À nommer pour situer ce que M4 couvre — et ce qu'il ne couvre pas.



BLOC 4.2

Sandboxing

Isoler le système de fichiers

Le sandbox confine les commandes shell lancées par l'agent (et leurs sous-processus), indépendamment du mode de permission choisi.

Écriture autorisée

Le répertoire de travail et ses sous-dossiers ; ailleurs, seulement les chemins ajoutés explicitement (`allowWrite`)

Lecture restreinte possible

Par défaut tout est lisible : bloquer les chemins sensibles (`denyRead`)

Chemins protégés sur Argos

```
// .claude/settings.json
"permissions": { "deny": ["Edit(/.env)"] },
"sandbox": {
  "enabled": true,
  "filesystem": {
    "denyWrite": ["/migrations", "/.env"]
  }
}
```

Le sandbox s'impose aux commandes shell au niveau de l'OS ; les outils d'édition de Claude relèvent des règles de permission. Les deux se combinent.



BLOC 4.3

Sécurité réseau

Restreindre l'accès réseau

```
"sandbox": {  
  "enabled": true,  
  "network": {  
    "allowedDomains": [  
      "registry.npmjs.org",  
      "api.notifications-argos.example"  
    ]  
  }  
}
```

Hors liste, Claude Code demande l'autorisation. Refus systématique : strictAllowlist, dans les réglages utilisateur ou gérés (ignoré dans ceux du dépôt).

En contexte d'entreprise

Proxy

Toutes les requêtes passent par un point de sortie unique, journalisé

mTLS

Authentification mutuelle entre le poste et l'infrastructure interne

Ces mécanismes se combinent avec l'allowlist : le proxy ne dispense pas de restreindre les domaines.



BLOC 4.4

Credentials

Ne jamais écrire un secret en clair

Variables d'environnement

La clé vit hors du dépôt, injectée au démarrage

Secret hors de portée de l'agent

Règle Read(/.env) en deny ; sandbox.credentials retire ou masque la variable (réglages utilisateur)

Sur Argos : la clé de notification

```
// avant
const key = "ntf_live_8f3e2b9c...";

// après
const key = process.env.NOTIF_API_KEY;
```

La clé retirée de l'historique Git est une étape distincte, hors périmètre de ce lab : elle suppose de réécrire l'historique du dépôt.



BLOC 4.5

Sécurité MCP

Un serveur MCP est une surface d'attaque

Un serveur MCP tiers peut retourner du contenu conçu pour manipuler l'agent : c'est l'injection de prompt.

- Ne connecter que des serveurs MCP dont la provenance est vérifiée
- Limiter les serveurs disponibles à ceux réellement nécessaires à la tâche
- Rester attentif à une instruction qui semble provenir du contenu récupéré, pas de l'utilisateur

Le moindre privilège, par défaut

Le principe qui résume ce module

N'accorder que l'accès strictement nécessaire à la tâche en cours : le mode de permission le plus restrictif qui permette d'avancer, l'allowlist réseau la plus courte possible, et jamais de bypass sur du code de production.



LAB M4

Fermer la faille

Brief

SOCLE — pour tous

Configurer un sandbox restrictif avec allowlist réseau sur Argos, et sortir la clé de notification du code source.

EXTENSION — pour aller plus loin

Masquer la clé dans les journaux de session, et restreindre les serveurs MCP connectés au strict nécessaire.

Pièges et repères

- Une allowlist trop large revient à ne pas en avoir : la limiter aux domaines réellement utilisés
- Retirer la clé du fichier sans la retirer de l'historique Git laisse la faille ouverte pour quiconque clone le dépôt
- Un sandbox mal testé peut bloquer une action légitime : vérifier après configuration, pas seulement avant

État attendu en fin de module

- Argos tourne avec un sandbox restrictif et une allowlist réseau
- La clé de notification n'apparaît plus en clair dans le code
- Les serveurs MCP connectés sont limités au strict nécessaire

Resynchronisation avant de passer à M5 — Environnements reproductibles.

Ce qu'il faut retenir de M4

Le moindre privilège se décline à chaque niveau : le mode de permission, le système de fichiers, le réseau, les credentials, les serveurs MCP.

Prochaine étape — M5

Argos est maintenant durci. Reste à garantir que cet environnement se reproduit à l'identique pour toute l'équipe : devcontainer et Codespaces en M5.



MODULE M5

Environnements reproductibles

Claude & Claude Code

Fil rouge : Argos

Ce que couvre M5

- | | | |
|---|-------------------------------|---|
| 1 | Dev containers | Intégrer Claude Code, persister l'authentification |
| 2 | GitHub Codespaces | Lancer Claude Code dans un environnement cloud préconfiguré |
| 3 | Comparatif | Sandbox local, devcontainer, VM, cloud : quand choisir quoi |
| 4 | Claude Code on the web | Bascule terminal ↔ web |

+ Lab : devcontainer avec Claude Code sur Argos, reproduit dans un Codespace

Durci, mais sur un seul poste

Acquis (M1-M4)

Argos tourne sous sandbox restrictif, sans secret en clair, avec une allowlist réseau.

Ce que ça ne garantit pas encore

Rien ne dit que cette configuration se retrouve à l'identique sur le poste d'un collègue, ou dans un environnement cloud.

À la fin de ce module

- Intégrer Claude Code dans un devcontainer
- Faire persister l'authentification entre deux reconstructions du conteneur
- Lancer le même environnement dans un GitHub Codespace
- Situer sandbox local, devcontainer, VM et cloud les uns par rapport aux autres
- Basculer une session entre le terminal et le web



BLOC 5.1

Dev containers

Un environnement versionné avec le projet

Un devcontainer décrit, dans un fichier du dépôt, l'image et les outils nécessaires pour développer sur le projet. Quiconque l'ouvre obtient le même environnement.

Sur Argos, cela règle un problème concret : la configuration de sécurité posée en M4 ne dépend plus du poste de chacun.

Intégrer Claude Code au devcontainer

```
// .devcontainer/devcontainer.json
{
  "image": "mcr.microsoft.com/devcontainers/javascript-node:22-bookworm",
  "features": {
    "ghcr.io/anthropics/devcontainer-features/claude-code:1.0": {}
  }
}
```

La feature officielle installe Claude Code ; garder la même version majeure de Node que le projet.

Ne pas se réauthentifier à chaque reconstruction

Reconstruire un conteneur perd tout ce qui n'est pas explicitement conservé. Sans précaution, il faut se reconnecter à chaque rebuild.

```
"mounts": [  
  "source=claude-config,target=/home/node/.claude,type=volume"  
],  
"containerEnv": { "CLAUDE_CONFIG_DIR": "/home/node/.claude" }
```

Sans CLAUDE_CONFIG_DIR, ~/.claude.json (le compte connecté) reste hors du volume et se perd au rebuild.

Ce que M4 devient dans le devcontainer

Permissions, hooks et CLAUDE.md versionnés s'appliquent tels quels. Le sandbox Bash, lui, ne démarre pas dans un conteneur Docker standard : l'allowlist réseau passe par un pare-feu du conteneur.

À vérifier par un essai de contournement : `/sandbox` signale l'absence de sandbox, et un curl vers un domaine hors liste doit échouer.



BLOC 5.2

GitHub Codespaces

Le même environnement, dans le cloud

Codespaces lance une machine cloud préconfigurée à partir du même devcontainer.json. Aucune installation locale n'est nécessaire au-delà d'un navigateur.

Ouvrir Argos dans un Codespace donne, en quelques minutes, exactement le même environnement que le devcontainer local.

Même spécification, exécution différente

Devcontainer local

Exécuté sur le poste, avec les ressources et le réseau de la machine locale.

Codespace

Exécuté sur une machine gérée par GitHub, accessible depuis n'importe quel poste.



BLOC 5.3

Comparatif des approches

Sandbox, devcontainer, VM, cloud

Sandbox local

Rapide, aucune installation

Dépend de la configuration du poste

Devcontainer

Reproductible, versionné avec le projet

Nécessite Docker sur le poste

VM

Isolation la plus forte

Plus lourd à démarrer et à maintenir

Cloud (Codespaces)

Zéro configuration poste, accessible partout

Dépend du réseau et d'un abonnement



BLOC 5.4

Claude Code on the web

Suivre une session depuis le navigateur

Remote Control (/remote-control) pilote depuis le navigateur une session du terminal. `claude --cloud` crée une session dans le cloud, que `claude --teleport` rapatrie dans le terminal avec son historique.

Cas d'usage typique

Lancer une tâche longue avec `claude --cloud` avant de partir, la suivre depuis le navigateur ou le mobile, puis la rapatrier avec `claude --teleport`.

Reproductibilité d'équipe

Un seul devcontainer.json

Versionné, identique pour toute l'équipe, revu comme le reste du code

Sans prompt de permission en CI

Un environnement automatisé ne peut pas répondre à une confirmation interactive



LAB M5

Reproduire Argos

Brief

SOCLE — pour tous

Écrire le devcontainer d'Argos avec Claude Code intégré et l'authentification persistante, puis ouvrir le même dépôt dans un Codespace.

EXTENSION — pour aller plus loin

Démarrer une tâche depuis le terminal du Codespace et la poursuivre depuis Claude Code on the web.

Pièges et repères

- Un devcontainer qui n'installe pas exactement la même version de Node que le poste habituel peut faire échouer des tests silencieusement
- Oublier le volume d'authentification force une reconnexion à chaque reconstruction : vérifier après un rebuild volontaire
- Le sandbox et l'allowlist de M4 doivent être vérifiés à nouveau dans le devcontainer, pas seulement supposés hérités

État attendu en fin de module

- Argos se lance à l'identique en devcontainer local et en Codespace
- L'authentification persiste après une reconstruction du conteneur
- Le sandbox et l'allowlist réseau de M4 sont actifs dans les deux environnements

Resynchronisation avant de passer à M6 — Aller plus loin.

Ce qu'il faut retenir de M5

Un environnement versionné avec le projet se reproduit à l'identique, sur un poste comme dans le cloud, sans effort de configuration répété.

Prochaine étape — M6

Argos est corrigé, configuré, coordonné en multi-agents, sécurisé, reproductible. Dernière étape : une méthode disciplinée pour construire la fonctionnalité finale.



MODULE M6

Aller plus loin

Claude & Claude Code

Fil rouge : Argos

Ce que couvre M6

- 1 Superpowers**
Une méthodologie disciplinée : brainstorming, TDD, débogage en 4 phases
- 2 Cycle complet en pratique**
Dérouter la méthode sur la dernière fonctionnalité d'Argos
- 3 Pour aller plus loin**
LLM local, BMAD-METHOD — démonstrations courtes
- 4 Synthèse**
Bilan des trois jours et de la trajectoire d'Argos

+ Lab : construire la recherche de tickets sur Argos en cycle complet

Une dernière fonctionnalité, construite autrement

Acquis (M1-M5)

Argos est corrigé, configuré, coordonné en multi-agents, sécurisé, reproductible.

Objectif de M6

Ajouter la recherche de tickets par mot-clé — non pas en improvisant, mais selon un cycle brainstorm → plan → TDD → revue.

À la fin de ce module

- Situer Superpowers comme méthodologie, distincte du produit Claude Code lui-même
- Mener un brainstorming socratique avant d'écrire du code
- Dérouler un cycle TDD red-green-refactor complet
- Appliquer un débogage en 4 phases plutôt qu'une correction au hasard
- Avoir livré la recherche de tickets sur Argos selon ce cycle



BLOC 6.1

Superpowers

Une méthodologie, pas un produit Anthropic

Superpowers est un framework communautaire, distribué comme plugin. Il encode des pratiques de développement disciplinées : brainstorming avant code, TDD, débogage structuré.

À garder en tête

Framework tiers : versionné et maintenu hors du contrôle d'Anthropic, à traiter comme tel dans un usage en production.

Brainstorming avant code

Avant d'écrire la moindre ligne, la méthode fait poser les questions qui clarifient le périmètre réel de la demande.

- La recherche porte-t-elle sur le titre du ticket, sa description, ou les deux ?
- Doit-elle être sensible à la casse ? Aux accents ?
- Que renvoyer si aucun ticket ne correspond ?

Red, green, refactor

1

Rouge

Écrire un test qui échoue, pour la fonctionnalité pas encore écrite

2

Vert

Écrire le minimum de code qui fait passer ce test

3

Refactor

Améliorer le code sans changer son comportement observable

Le cycle se répète, une petite fonctionnalité à la fois, jamais en écrivant tout le code avant le premier test.

Débogage en 4 phases

1

Reproduire

Isoler un cas qui déclenche le défaut de façon fiable

2

Isoler

Réduire au plus petit périmètre qui reproduit encore le défaut

3

Corriger

Modifier la cause identifiée, pas seulement le symptôme

4

Vérifier

Confirmer par le test, pas seulement par observation

Installation

```
/plugin marketplace add obra/superpowers-marketplace  
/plugin install superpowers@superpowers-marketplace
```

Aussi dans la marketplace officielle : /plugin install superpowers@claude-plugins-official.



BLOC 6.2

Cycle complet en pratique

La recherche de tickets, étape par étape

Brainstorm

Clarifier le périmètre : champs concernés, casse, résultat vide

Plan

Décomposer en tests à écrire avant toute implémentation

TDD

Red, green, refactor sur chaque cas identifié

Revue

Relecture par le sub-agent reviewer créé en M3

Quand se discipliner ainsi

Cycle complet justifié

Logique métier non triviale, destinée à durer et à être maintenue par d'autres.

Formalisme excessif

Prototype jetable ou exploration ponctuelle : le coût du cycle complet dépasse le bénéfice.



POUR ALLER PLUS LOIN

Deux démonstrations

Ollama et modèles ouverts face au cloud

Confidentialité

Les données ne quittent jamais le poste ou le réseau interne

Performance

Capacité généralement en retrait par rapport à un modèle cloud à jour

Démonstration courte — un compromis à arbitrer selon la sensibilité des données traitées, pas une réponse universelle.

Des personas agiles, un cycle imposé

Framework tiers multi-agents avec des rôles fixes (Analyst, PM, Architect, Dev, QA) et un cycle Analyse → Planification → Solutioning → Implémentation.

Face aux agent teams natifs

BMAD impose une structure de rôles métier fixe ; les agent teams vus en M3 sont plus légers et se recomposent librement selon la tâche.



LAB M6

Construire la recherche

Brief

SOCLE — pour tous

Construire la recherche de tickets par mot-clé sur Argos selon le cycle brainstorm → plan → TDD → revue.

EXTENSION — pour aller plus loin

Tester un modèle local sur une sous-tâche simple d'Argos, ou explorer BMAD-METHOD sur un cas fictif court.

Pièges et repères

- Écrire l'implémentation avant le test annule l'intérêt du cycle : le test rouge doit précéder le code
- Un refactor qui change le comportement observable n'en est plus un : le vérifier par les tests, pas à l'œil
- Sauter le brainstorming sur une fonctionnalité qui semble simple est la cause la plus fréquente de retour en arrière

État attendu en fin de module

- La recherche de tickets est livrée, testée, et relue par le sub-agent
- Chaque étape du cycle (brainstorm, plan, TDD, revue) a été suivie explicitement
- La suite de tests complète d'Argos passe toujours

Le parcours d'Argos

M1

Bug de filtrage corrigé, premiers pas avec l'agent

M2

Conventions consignées : CLAUDE.md, commande, hook

M3

Commentaires livrés par une équipe d'agents coordonnée

M4

Faible de sécurité corrigée, accès restreints

M5

Environnement reproductible, local et cloud

M6

Recherche construite selon un cycle TDD discipliné

Ce qui reste après la formation

- Utiliser Claude Code en autonomie, du diagnostic à la vérification
- Configurer et personnaliser un projet pour l'agent, une fois pour toutes
- Coordonner plusieurs agents sans conflit sur une tâche réelle
- Sécuriser un usage agentique en production
- Rendre un environnement de développement reproductible et discipliné

Formation terminée

Claude & Claude Code — maîtriser le développement agentique

Merci d'avoir suivi Argos jusqu'au bout.

